

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyse	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate
7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze

8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

Code of Conduct:

I A i s h w a r y a / 1 9 2 4 2 1 4 3 2 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1) Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used.

ANS

```
MySQL 8.0 Command Line Cli x + v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE Department;
Query OK, 1 row affected (0.12 sec)

mysql> USE Department;
Database changed
mysql> CREATE TABLE Department(DeptID INT PRIMARY KEY,DeptName VARCHAR(30));
Query OK, 0 rows affected (0.18 sec)

mysql> CREATE TABLE Student(StudentID INT PRIMARY KEY,Name VARCHAR(30),DeptID INT,Marks INT,FOREIGN KEY(DeptID) REFERENCES Department(DeptID));
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '(30),DeptID INT,Marks INT,FOREIGN KEY(DeptID) REFERENCES Department(DeptID))' at line 1
mysql> CREATE TABLE Student(StudentID INT PRIMARY KEY,Name VARCHAR(30),DeptID INT,Marks INT,FOREIGN KEY(DeptID) REFERENCES Department(DeptID));
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'KEY(DeptID) REFERENCES Department(DeptID))' at line 1
mysql> CREATE TABLE Student(StudentID INT PRIMARY KEY,Name VARCHAR(30),DeptID INT,Marks INT,FOREIGN KEY(DeptID) REFERENCES Department(DeptID));
Query OK, 0 rows affected (0.11 sec)

mysql> INSERT INTO Department VALUES(10,'CSE'),(20,'ECE'),(30,'EEE');
Query OK, 3 rows affected (0.04 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Department
-> ;
+-----+-----+
| DeptID | DeptName |
+-----+-----+
| 10     | CSE      |
| 20     | ECE      |
| 30     | EEE      |
+-----+-----+
3 rows in set (0.02 sec)

mysql> INSERT INTO Student VALUES(101,'Rahul',10,85),(102,'Priya',20,90),(103,'Arun',10,75),(104,'Sneha',30,80);
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '(10,75),(104,'Sneha',30,80)' at line 1
mysql> INSERT INTO Student VALUES(101,'Rahul',10,85),(102,'Priya',20,90),(103,'Arun',10,75),(104,'Sneha',30,80);
Query OK, 4 rows affected (0.05 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Student VALUES(101,'Rahul',10,85),(102,'Priya',20,90),(103,'Arun',10,75),(104,'Sneha',30,80);
ERROR 1062 (23000): Duplicate entry '101' for key 'student.PRIMARY'
mysql> INSERT INTO Student VALUES(101,'Rahul',10,85),(102,'Priya',20,90),(103,'Arun',10,75),(104,'Sneha',30,80);
ERROR 1062 (23000): Duplicate entry '101' for key 'student.PRIMARY'
mysql> SELECT * FROM Student;
+-----+-----+-----+-----+
| StudentID | Name  | DeptID | Marks |
+-----+-----+-----+-----+
| 101       | Rahul | 10     | 85    |
| 102       | Priya | 20     | 90    |
| 103       | Arun  | 10     | 75    |
| 104       | Sneha | 30     | 80    |
+-----+-----+-----+-----+
4 rows in set (0.02 sec)

mysql> SELECT Student.Name,Department,DeptName FROM Student INNER JOIN Department ON Student.DeptID = Department.DeptID;
ERROR 1054 (42S22): Unknown column 'Department' in 'field list'
mysql> SELECT Student.Name,Department.DeptName FROM Student INNER JOIN Department ON Student.DeptID = Department.DeptID;
+-----+-----+
| Name  | DeptName |
+-----+-----+
| Rahul | CSE      |
| Arun  | CSE      |
| Priya | ECE      |
| Sneha | EEE      |
+-----+-----+
4 rows in set (0.01 sec)
```

Logic

- The innermost query calculates the average marks.
- The second query finds the StudentIDs whose marks are greater than the average.
- The outer query retrieves the student names corresponding to those IDs.

2) Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.

ANS:

Relations

STUDENT(StudentID, Name, DeptID)

StudentID	Name	DeptID
101	Aishwarya	D1
102	Rahul	D2
103	Priya	D1

DEPARTMENT(DeptID, DeptName)

DeptID	DeptName
D1	Information Technology
D2	Computer Science

MARKS(StudentID, Subject, Marks)

StudentID	Subject	Marks
101	DBMS	85
102	DBMS	78
103	DBMS	92

Query 1

Retrieve the names of students who belong to the Information Technology department.

Relational Algebra

$$\pi_{Name} \left(\sigma_{DeptName='Information\ Technology'} (STUDENT \bowtie DEPARTMENT) \right)$$

Result

Name
Aishwarya
Priya

Query 2

Retrieve student names and marks of students who scored more than 80.

Relational Algebra

$$\pi_{Name, Marks}(\sigma_{Marks > 80}(STUDENT \bowtie MARKS))$$

Result

Name	Marks
Aishwarya	85
Priya	92

Query 3

Retrieve names of students who belong to the Information Technology department and scored more than 80.

Relational Algebra

$$\pi_{Name}(\sigma_{DeptName = 'Information Technology' \wedge Marks > 80}(STUDENT \bowtie DEPARTMENT \bowtie MARKS))$$

Result

Name
Aishwarya
Priya

Query 4

Retrieve the department names of students who scored more than 90.

Relational Algebra

$$\pi_{DeptName}(\sigma_{Marks > 90}(DEPARTMENT \bowtie STUDENT \bowtie MARKS))$$

Result

DeptName
Information Technology

Operators Used

Operator	Meaning
σ (Selection)	Selects rows that satisfy a condition
π (Projection)	Selects specific columns
$\bowtie$ (Join)	Combines tuples from related tables

Conclusion

- Selection (σ) filters records based on conditions.
- Projection (π) retrieves only the required attributes.
- Join ($\bowtie$) combines related relations using common attributes.
- Multiple conditions can be combined using AND ($\wedge$) or OR ($\vee$) in relational algebra expressions.

3) Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates.

ANS:

Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates.

Scenario: Student Management System

A college maintains student records, department details, and student marks.

Step 1: Design the Relational Schema

STUDENT

Attribute	Data Type
StudentID	INT (Primary Key)
Name	VARCHAR(50)
DeptID	VARCHAR(10)

DEPARTMENT

Attribute	Data Type
DeptID	VARCHAR(10) (Primary Key)
DeptName	VARCHAR(50)

MARKS

Attribute	Data Type
StudentID	INT
Subject	VARCHAR(30)
Marks	INT

```
MySQL 8.0 Command Line CL  x  +  v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 13
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE Student;
Query OK, 1 row affected (0.12 sec)

mysql> USE Student;
Database changed
mysql> CREATE TABLE Student( StudentID INT PRIMARY KEY, Name VARCHAR(50), DeptID VARCHAR(10));
Query OK, 0 rows affected (0.12 sec)

mysql> CREATE TABLE Department( DeptID VARCHAR(10) PRIMARY KEY, DeptName VARCHAR(50));
Query OK, 0 rows affected (0.08 sec)

mysql> CREATE TABLE Marks( StudentID INT, Subject VARCHAR(30), Marks INT );
Query OK, 0 rows affected (0.08 sec)

mysql> INSERT INTO Department VALUES ('D1','Information Technology'), ('D2','Computer Science');
Query OK, 2 rows affected (0.06 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Marks VALUES(101,'DBMS',85),(102,'DBMS',78),(103,'DBMS',92);
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
MySQL 8.0 Command Line CL  x  +  v

mysql> SELECT S.Name, D.DeptName, M.Marks FROM Student S JOIN Department D ON S.DeptID = D.DeptID JOIN Marks M ON S.StudentID = M.StudentID;
Empty set (0.00 sec)

mysql> SELECT * FROM Student;
Empty set (0.04 sec)

mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName |
+-----+-----+
| D1     | Information Technology |
| D2     | Computer Science |
+-----+-----+
2 rows in set (0.04 sec)

mysql> SELECT * FROM Marks;
+-----+-----+-----+
| StudentID | Subject | Marks |
+-----+-----+-----+
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
+-----+-----+-----+
3 rows in set (0.04 sec)

mysql> INSERT INTO Student VALUES
-> (101,'Aishwarya','D1'),
-> (102,'Rahul','D2'),
-> (103,'Priya','D1');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Student;
+-----+-----+-----+
| StudentID | Name      | DeptID |
+-----+-----+-----+
| 101       | Aishwarya | D1     |
| 102       | Rahul     | D2     |
| 103       | Priya     | D1     |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
MySQL 8.0 Command Line CL  x  +  v

mysql> SELECT S.Name, D.DeptName, M.Marks
-> FROM Student S
-> JOIN Department D
-> ON S.DeptID = D.DeptID
-> JOIN Marks M
-> ON S.StudentID = M.StudentID;
+-----+-----+-----+
| Name      | DeptName | Marks |
+-----+-----+-----+
| Aishwarya | Information Technology | 85    |
| Rahul     | Computer Science | 78    |
| Priya     | Information Technology | 92    |
+-----+-----+-----+
3 rows in set (0.04 sec)

mysql> UPDATE MARKS
-> SET Marks = 85
-> WHERE StudentID = 102;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM MARKS;
+-----+-----+-----+
| StudentID | Subject | Marks |
+-----+-----+-----+
| 101       | DBMS    | 85    |
| 102       | DBMS    | 85    |
| 103       | DBMS    | 92    |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> DELETE FROM MARKS
-> WHERE StudentID = 102;
Query OK, 1 row affected (0.05 sec)
```

```
mysql> SELECT * FROM MARKS;
+-----+-----+-----+
| StudentID | Subject | Marks |
+-----+-----+-----+
| 101       | DBMS    | 85    |
| 103       | DBMS    | 92    |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```



Explanation

- **Relational Schema:** Defines the structure of the database using tables and attributes.
- **CREATE TABLE:** Creates the required tables.
- **INSERT:** Adds records to the tables.
- **SELECT:** Retrieves the required data.
- **UPDATE:** Modifies existing data.
- **DELETE:** Removes a record from the table.

4) Analyze a given SQL query with nested subqueries and predict its output, explaining each step of execution.

ANS:

```
MySQL 8.0 Command Line Cl...
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 19
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE CollegeDB;
Query OK, 1 row affected (0.05 sec)

mysql> USE CollegeDB;
Database changed
mysql> CREATE TABLE Student(
  -> StudentID INT PRIMARY KEY,
  -> Name VARCHAR(50),
  -> DeptID VARCHAR(10)
  -> );
Query OK, 0 rows affected (0.09 sec)

mysql> CREATE TABLE Marks(
  -> StudentID INT,
  -> Subject VARCHAR(30),
  -> Marks INT
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Student VALUES
  -> (101,'Aishwarya','D1'),
  -> (102,'Rahul','D2'),
  -> (103,'Priya','D1');
Query OK, 3 rows affected (0.05 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
MySQL 8.0 Command Line Cl...

mysql> INSERT INTO Student VALUES
  -> (101,'Aishwarya','D1'),
  -> (102,'Rahul','D2'),
  -> (103,'Priya','D1');
Query OK, 3 rows affected (0.05 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Marks VALUES
  -> (101,'DBMS',85),
  -> (102,'DBMS',78),
  -> (103,'DBMS',92);
Query OK, 3 rows affected (0.05 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Student;
+-----+-----+-----+
| StudentID | Name      | DeptID |
+-----+-----+-----+
| 101       | Aishwarya | D1     |
| 102       | Rahul     | D2     |
| 103       | Priya     | D1     |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Marks;
+-----+-----+-----+
| StudentID | Subject | Marks |
+-----+-----+-----+
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT Name
  -> FROM Student
  -> WHERE StudentID IN
  -> (
  -> SELECT StudentID
  -> FROM Marks
```

```
MySQL 8.0 Command Line C... X + -
-> WHERE Marks >
-> (
-> SELECT AVG(Marks)
-> FROM Marks
-> )
-> );
+-----+
| Name |
+-----+
| Priya |
+-----+
1 row in set (0.05 sec)

mysql> SELECT AVG(Marks)
-> FROM Marks;
+-----+
| AVG(Marks) |
+-----+
| 85.0000 |
+-----+
1 row in set (0.04 sec)

mysql> SELECT StudentID
-> FROM Marks
-> WHERE Marks > 85;
+-----+
| StudentID |
+-----+
| 103 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT Name
-> FROM Student
-> WHERE StudentID IN (103);
+-----+
| Name |
+-----+
| Priya |
+-----+
1 row in set (0.00 sec)
```

Logic Used

- The inner query finds the maximum marks.
- The outer query returns the student whose marks match the maximum value.

Conclusion

- JOIN is used to combine data from multiple related tables using common columns.
- Subqueries are queries written inside another query to perform intermediate calculations or filtering.
- Together, joins and subqueries help retrieve meaningful information efficiently from relational databases.

5) Compare different SQL approaches (Joins vs Subqueries) for solving the same problem and evaluate their efficiency.

ANS:

Problem Statement

A college maintains student records, department details, and marks. Retrieve the student name, department name, and marks of students who scored more than 80 using both JOIN and Subquery approaches, and compare their efficiency.

```
MySQL 8.0 Command Line CL
```

```
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 20
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE CollegeDB;
ERROR 1007 (HY000): Can't create database 'collegedb'; database exists
mysql> USE CollegeDB;
Database changed
mysql> CREATE TABLE Student(
  -> StudentID INT PRIMARY KEY,
  -> Name VARCHAR(50),
  -> DeptID VARCHAR(10)
  -> );
ERROR 1050 (42S01): Table 'student' already exists
mysql> CREATE TABLE Department(
  -> DeptID VARCHAR(10) PRIMARY KEY,
  -> DeptName VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.20 sec)

mysql> CREATE TABLE Marks(
  -> StudentID INT,
  -> Subject VARCHAR(30),
  -> Marks INT
  -> );
ERROR 1050 (42S01): Table 'marks' already exists
mysql> INSERT INTO Student VALUES
  -> (101, 'Aishwarya', 'D1'),
  -> (102, 'Rahul', 'D2'),
  -> (103, 'Priya', 'D1');
ERROR 1062 (23000): Duplicate entry '101' for key 'student.PRIMARY'
mysql> INSERT INTO Department VALUES
  -> ('D1', 'Information Technology'),
  -> ('D2', 'Computer Science');
Query OK, 2 rows affected (0.03 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Marks VALUES
  -> (101, 'DBMS', 85),
  -> (102, 'DBMS', 78),
  -> (103, 'DBMS', 92);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Student;
+-----+-----+-----+
| StudentID | Name      | DeptID |
+-----+-----+-----+
| 101       | Aishwarya | D1     |
| 102       | Rahul     | D2     |
| 103       | Priya     | D1     |
+-----+-----+-----+
3 rows in set (0.01 sec)

mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName      |
+-----+-----+
| D1     | Information Technology |
| D2     | Computer Science   |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Marks;
+-----+-----+-----+
| StudentID | Subject | Marks |
+-----+-----+-----+
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
+-----+-----+-----+
6 rows in set (0.01 sec)
```

```
MySQL 8.0 Command Line Cl...
+-----+
6 rows in set (0.00 sec)

mysql> SELECT S.Name,
-> D.DeptName,
-> M.Marks
-> FROM Student S
-> JOIN Department D
-> ON S.DeptID = D.DeptID
-> JOIN Marks M
-> ON S.StudentID = M.StudentID
-> WHERE M.Marks > 80;
+-----+
| Name | DeptName | Marks |
+-----+
| Priya | Information Technology | 92 |
| Aishwarya | Information Technology | 85 |
| Priya | Information Technology | 92 |
| Aishwarya | Information Technology | 85 |
+-----+
4 rows in set (0.02 sec)

mysql> SELECT * FROM Student;
+-----+
| StudentID | Name | DeptID |
+-----+
| 101 | Aishwarya | D1 |
| 102 | Rahul | D2 |
| 103 | Priya | D1 |
+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Department;
+-----+
| DeptID | DeptName |
+-----+
| D1 | Information Technology |
| D2 | Computer Science |
+-----+
2 rows in set (0.00 sec)
```

```
MySQL 8.0 Command Line Cl...
+-----+
mysql> SELECT * FROM Marks;
+-----+
| StudentID | Subject | Marks |
+-----+
| 101 | DBMS | 85 |
| 102 | DBMS | 78 |
| 103 | DBMS | 92 |
| 101 | DBMS | 85 |
| 102 | DBMS | 78 |
| 103 | DBMS | 92 |
+-----+
6 rows in set (0.00 sec)

mysql> SELECT DISTINCT
-> S.Name,
-> D.DeptName,
-> M.Marks
-> FROM Student S
-> JOIN Department D
-> ON S.DeptID = D.DeptID
-> JOIN Marks M
-> ON S.StudentID = M.StudentID
-> WHERE M.Marks > 80;
+-----+
| Name | DeptName | Marks |
+-----+
| Priya | Information Technology | 92 |
| Aishwarya | Information Technology | 85 |
+-----+
2 rows in set (0.01 sec)

mysql> SELECT Name
-> FROM Student
-> WHERE StudentID IN
-> (
-> SELECT StudentID
-> FROM Marks
-> WHERE Marks > 80
-> );
+-----+
-> );
```

```
+-----+
| Name |
+-----+
| Aishwarya |
| Priya |
+-----+
2 rows in set (0.01 sec)

mysql>
```

Approach 1 – Using JOIN

Output

Name	DeptName	Marks
Aishwarya	Information Technology	85
Priya	Information Technology	92

Explanation

- JOIN combines related tables using common attributes.
- Student is joined with Department using DeptID.
- Student is joined with Marks using StudentID.
- The WHERE clause filters students who scored more than 80.

Approach 2 – Using Subquery

Output

Name
Aishwarya
Priya

Explanation

- The inner query retrieves the StudentID values of students with marks greater than 80.
- The outer query uses these IDs to display the student names.

Comparison of JOIN and Subquery

Feature	JOIN	Subquery
Purpose	Combines data from multiple tables	Retrieves data using one query inside another
Readability	Easier for multiple related tables	Easier for simple filtering
Performance	Faster for large datasets	May be slower due to nested execution
Data Retrieval	Retrieves columns from multiple tables	Mainly used for filtering results
Complexity	Suitable for complex relationships	Suitable for simple conditions
Efficiency	High	Moderate

Efficiency Evaluation

JOIN

Advantages

- Faster execution on large databases.
- Retrieves data from multiple tables in a single query.
- Preferred in real-world applications.

Disadvantages

- Query may become lengthy with many tables.

Subquery

Advantages

- Simple and easy to understand.
- Useful for filtering based on calculated values or intermediate results.

Disadvantages

- Can be slower than joins because the inner query may need to be executed before the outer query.
- Less efficient for complex multi-table operations.

Conclusion

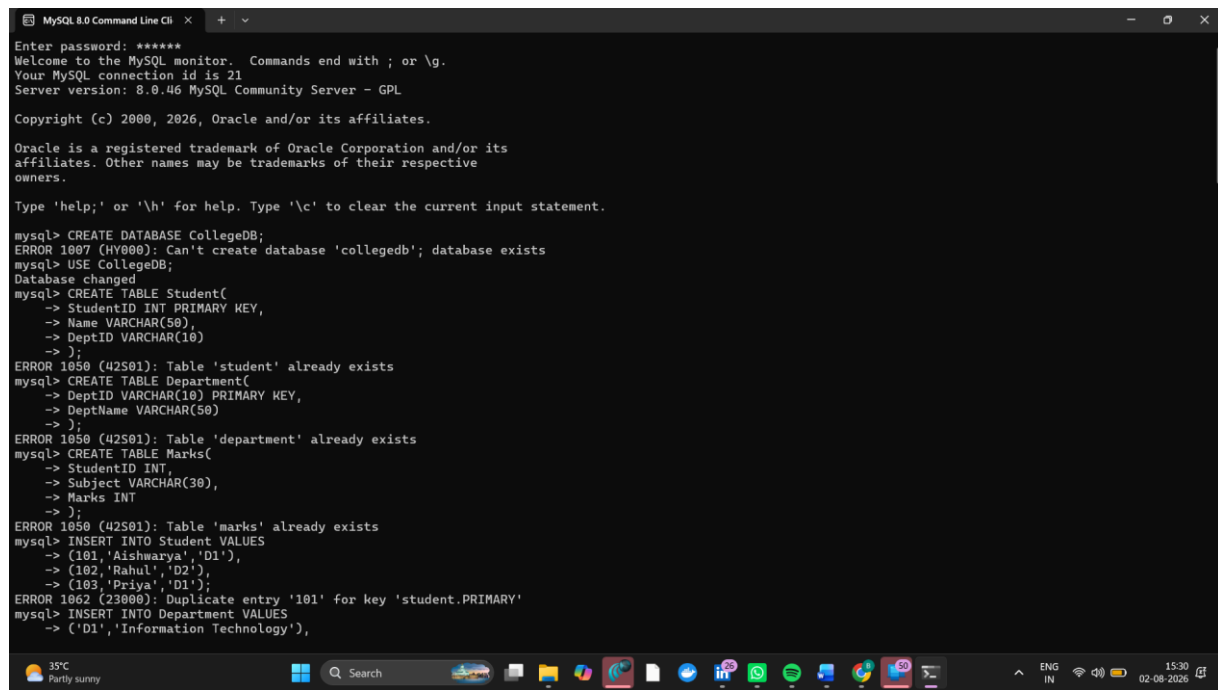
Both JOIN and Subquery can solve the same problem. However:

- JOIN is recommended for retrieving related data from multiple tables because it is generally faster, more efficient, and widely used in database applications.
- Subquery is recommended for simple filtering and calculations where intermediate results are needed.

Therefore, JOIN is the preferred and more efficient solution for this scenario.

6) Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction.

ANS



```
MySQL 8.0 Command Line Client
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 21
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE CollegeDB;
ERROR 1007 (HY000): Can't create database 'collegedb'; database exists
mysql> USE CollegeDB;
Database changed
mysql> CREATE TABLE Student(
  -> StudentID INT PRIMARY KEY,
  -> Name VARCHAR(50),
  -> DeptID VARCHAR(10)
  -> );
ERROR 1050 (42S01): Table 'student' already exists
mysql> CREATE TABLE Department(
  -> DeptID VARCHAR(10) PRIMARY KEY,
  -> DeptName VARCHAR(50)
  -> );
ERROR 1050 (42S01): Table 'department' already exists
mysql> CREATE TABLE Marks(
  -> StudentID INT,
  -> Subject VARCHAR(30),
  -> Marks INT
  -> );
ERROR 1050 (42S01): Table 'marks' already exists
mysql> INSERT INTO Student VALUES
  -> (101, 'Aishwarya', 'D1'),
  -> (102, 'Rahul', 'D2'),
  -> (103, 'Priya', 'D1');
ERROR 1062 (23000): Duplicate entry '101' for key 'student.PRIMARY'
mysql> INSERT INTO Department VALUES
  -> ('D1', 'Information Technology');
```

```
MySQL 8.0 Command Line CL> +
-> ('D2','Computer Science');
ERROR 1062 (23000): Duplicate entry 'D1' for key 'department.PRIMARY'
mysql> INSERT INTO Marks VALUES
-> (101,'DBMS',85),
-> (102,'DBMS',78),
-> (103,'DBMS',92);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Student;
+-----+-----+-----+
| StudentID | Name   | DeptID |
+-----+-----+-----+
| 101       | Aishwarya | D1     |
| 102       | Rahul    | D2     |
| 103       | Priya    | D1     |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName |
+-----+-----+
| D1     | Information Technology |
| D2     | Computer Science |
+-----+-----+
2 rows in set (0.04 sec)

mysql> SELECT * FROM Marks;
+-----+-----+-----+
| StudentID | Subject | Marks |
+-----+-----+-----+
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
+-----+-----+-----+
9 rows in set (0.00 sec)
```

```
MySQL 8.0 Command Line CL> +
+-----+-----+-----+
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
+-----+-----+-----+
9 rows in set (0.00 sec)

mysql> CREATE VIEW Student_View AS
-> SELECT S.Name,
-> D.DeptName,
-> M.Marks
-> FROM Student S
-> JOIN Department D
-> ON S.DeptID = D.DeptID
-> JOIN Marks M
-> ON S.StudentID = M.StudentID;
Query OK, 0 rows affected (0.09 sec)

mysql> SELECT * FROM Student_View;
+-----+-----+-----+
| Name   | DeptName | Marks |
+-----+-----+-----+
| Aishwarya | Information Technology | 85    |
| Rahul    | Computer Science | 78    |
| Priya    | Information Technology | 92    |
| Aishwarya | Information Technology | 85    |
| Rahul    | Computer Science | 78    |
| Priya    | Information Technology | 92    |
| Aishwarya | Information Technology | 85    |
| Rahul    | Computer Science | 78    |
| Priya    | Information Technology | 92    |
+-----+-----+-----+
9 rows in set (0.01 sec)

mysql> SELECT * FROM Student;
+-----+-----+-----+
| StudentID | Name   | DeptID |
+-----+-----+-----+
| 101       | Aishwarya | D1     |
| 102       | Rahul    | D2     |
| 103       | Priya    | D1     |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName |
+-----+-----+
| D1     | Information Technology |
| D2     | Computer Science |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Marks;
+-----+-----+-----+
| StudentID | Subject | Marks |
+-----+-----+-----+
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
+-----+-----+-----+
9 rows in set (0.00 sec)

mysql> DROP VIEW Student_View;
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE VIEW Student_View AS
-> SELECT DISTINCT
->     S.Name,
->     D.DeptName,
->     M.Marks
-> FROM Student S
-> JOIN Department D
-> ON S.DeptID = D.DeptID
-> JOIN Marks M
-> ON S.StudentID = M.StudentID;
Query OK, 0 rows affected (0.05 sec)

mysql> SELECT * FROM Student_View;
+-----+-----+-----+
| Name      | DeptName | Marks |
+-----+-----+-----+
| Aishwarya | Information Technology | 85    |
| Rahul     | Computer Science | 78    |
| Priya     | Information Technology | 92    |
+-----+-----+-----+
3 rows in set (0.01 sec)
```

Justification for Using Views

1. Security

- Users can access only the required information.
- Sensitive data such as StudentID is hidden from unauthorized users.

2. Data Abstraction

- Users work with a simplified virtual table instead of multiple base tables.
- The complexity of SQL joins is hidden.

3. Easy Data Access

- Frequently used queries are saved as a view.
- Users can retrieve data with a simple `SELECT * FROM Student_View;`

4. Improved Maintainability

- Changes to the underlying tables can often be managed without changing how users query the view.

Advantages of Views

- Restricts unauthorized access to sensitive data.
- Simplifies complex SQL queries.
- Improves data security.
- Provides logical data independence.
- Makes reporting easier.

7) Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation.

ANS

Problem Statement

Consider a **Student Management System** with the following database schema:

STUDENT

StudentID	Name	DeptID
101	Aishwarya	D1
102	Rahul	D2
103	Priya	D1

DEPARTMENT

DeptID	DeptName
D1	Information Technology
D2	Computer Science

MARKS

StudentID	Subject	Marks
101	DBMS	85
102	DBMS	78
103	DBMS	92

```
MySQL 8.0 Command Line Client
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 22
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE CollegeDB;
ERROR 1007 (HY000): Can't create database 'collegedb'; database exists
mysql> USE CollegeDB;
Database changed
mysql> CREATE TABLE Student(
  -> StudentID INT PRIMARY KEY,
  -> Name VARCHAR(50),
  -> DeptID VARCHAR(10)
  -> );
ERROR 1050 (42S01): Table 'student' already exists
mysql> CREATE TABLE Department(
  -> DeptID VARCHAR(10) PRIMARY KEY,
  -> DeptName VARCHAR(50)
  -> );
ERROR 1050 (42S01): Table 'department' already exists
mysql> CREATE TABLE Marks(
  -> StudentID INT,
  -> Subject VARCHAR(30),
  -> Marks INT
  -> );
ERROR 1050 (42S01): Table 'marks' already exists
mysql> INSERT INTO Student VALUES
  -> (101,'Aishwarya','D1'),
  -> (102,'Rahul','D2'),
  -> (103,'Priya','D1');
ERROR 1062 (23000): Duplicate entry '101' for key 'student.PRIMARY'
mysql> INSERT INTO Department VALUES
  -> ('D1','Information Technology'),
```

```
MySQL 8.0 Command Line CLi x + v
-> ('D2','Computer Science');
ERROR 1062 (23000): Duplicate entry 'D1' for key 'department.PRIMARY'
mysql> INSERT INTO Marks VALUES
-> (101,'DBMS',85),
-> (102,'DBMS',78),
-> (103,'DBMS',92);
Query OK, 3 rows affected (0.02 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Student;
+-----+-----+-----+
| StudentID | Name      | DeptID |
+-----+-----+-----+
| 101       | Aishwarya | D1     |
| 102       | Rahul     | D2     |
| 103       | Priya     | D1     |
+-----+-----+-----+
3 rows in set (0.04 sec)

mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName      |
+-----+-----+
| D1     | Information Technology |
| D2     | Computer Science      |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Marks;
+-----+-----+-----+
| StudentID | Subject | Marks |
+-----+-----+-----+
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
+-----+-----+-----+
12 rows in set (0.00 sec)
```

```
MySQL 8.0 Command Line CLi x + v
+-----+-----+-----+
| StudentID | Subject | Marks |
+-----+-----+-----+
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
+-----+-----+-----+
12 rows in set (0.00 sec)

mysql> SELECT S.Name,
-> D.DeptName,
-> M.Marks,
-> FROM Student S
-> JOIN Department D
-> ON S.DeptID = D.DeptID
-> JOIN Marks M
-> ON S.StudentID = M.StudentID
-> WHERE M.Marks > 80;
+-----+-----+-----+
| Name      | DeptName      | Marks |
+-----+-----+-----+
| Aishwarya | Information Technology | 85    |
| Priya     | Information Technology | 92    |
| Aishwarya | Information Technology | 85    |
| Priya     | Information Technology | 92    |
| Aishwarya | Information Technology | 85    |
| Priya     | Information Technology | 92    |
| Aishwarya | Information Technology | 85    |
| Priya     | Information Technology | 92    |
+-----+-----+-----+
8 rows in set (0.02 sec)

mysql> SELECT * FROM Student;
+-----+-----+-----+
| StudentID | Name      | DeptID |
+-----+-----+-----+
| 101       | Aishwarya | D1     |
| 102       | Rahul     | D2     |
| 103       | Priya     | D1     |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
MySQL 8.0 Command Line CLi x + v

mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName      |
+-----+-----+
| D1     | Information Technology |
| D2     | Computer Science      |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Marks;
+-----+-----+-----+
| StudentID | Subject | Marks |
+-----+-----+-----+
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
+-----+-----+-----+
12 rows in set (0.00 sec)

mysql> SELECT DISTINCT
-> S.Name,
-> D.DeptName,
-> M.Marks
-> FROM Student S
-> JOIN Department D
-> ON S.DeptID = D.DeptID
-> JOIN Marks M
-> ON S.StudentID = M.StudentID
-> WHERE M.Marks > 80;
```

```

-> ON S.StudentID = M.StudentID
-> WHERE M.Marks > 80;

```

Name	DeptName	Marks
Aishwarya	Information Technology	85
Priya	Information Technology	92

2 rows in set (0.04 sec)

Relational Algebra Expression

$$\pi_{Name, DeptName, Marks}(\sigma_{Marks > 80}(Student \bowtie Department \bowtie Marks))$$

Explanation

- **σ (Selection):** Selects students whose marks are greater than 80.
- **$\bowtie$ (Join):** Combines the Student, Department, and Marks relations.
- **π (Projection):** Displays only Name, Department Name, and Marks.

Analysis of Differences

Relational Algebra	SQL
Procedural query language	Declarative query language
Uses symbols such as σ , π , $\bowtie$	Uses SQL keywords such as SELECT, FROM, WHERE, JOIN
Describes the sequence of operations	Describes the required result without specifying execution order
Mainly used for theoretical database design	Used in real-world database management systems
Mathematical notation	English-like syntax
Easier for query optimization analysis	Easier to write and understand for programmers

Comparison

Relational Algebra

Advantages

- Provides a mathematical foundation for database operations.
- Useful for analyzing and optimizing queries.
- Clearly shows the sequence of operations.

Disadvantages

- Difficult to understand for beginners.
- Not directly executable in database systems.

SQL

Advantages

- Easy to write and understand.
- Directly executable in MySQL, Oracle, SQL Server, etc.
- Supports complex queries, functions, and constraints.

Disadvantages

- Does not explicitly show the internal execution process.
- Query optimization is handled automatically by the DBMS.

8) Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements.

ANS

```
MySQL 8.0 Command Line Cli x + v
Enter password: *****
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 23
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE CollegeDB;
ERROR 1007 (HY000): Can't create database 'collegedb'; database exists
mysql> USE CollegeDB;
Database changed
mysql> CREATE TABLE Student(
  -> StudentID INT PRIMARY KEY,
  -> Name VARCHAR(50),
  -> DeptID VARCHAR(10)
  -> );
ERROR 1050 (42S01): Table 'student' already exists
mysql> CREATE TABLE Department(
  -> DeptID VARCHAR(10) PRIMARY KEY,
  -> DeptName VARCHAR(50)
  -> );
ERROR 1050 (42S01): Table 'department' already exists
mysql> CREATE TABLE Marks(
  -> StudentID INT,
  -> Subject VARCHAR(30),
  -> Marks INT
  -> );
ERROR 1050 (42S01): Table 'marks' already exists
mysql> INSERT INTO Student VALUES
  -> (101, 'Aishwarya', 'D1'),
  -> (102, 'Rahul', 'D2'),
  -> (103, 'Priya', 'D1');
ERROR 1062 (23000): Duplicate entry '101' for key 'student.PRIMARY'
mysql> INSERT INTO Department VALUES
  -> ('D1', 'Information Technology'),
```

```
  -> ('D2', 'Computer Science');
ERROR 1062 (23000): Duplicate entry 'D1' for key 'department.PRIMARY'
mysql> INSERT INTO Marks VALUES
  -> (101, 'DBMS', 85),
  -> (102, 'DBMS', 78),
  -> (103, 'DBMS', 92);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Student;
+-----+-----+-----+
| StudentID | Name      | DeptID |
+-----+-----+-----+
| 101       | Aishwarya | D1     |
| 102       | Rahul     | D2     |
| 103       | Priya     | D1     |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName      |
+-----+-----+
| D1     | Information Technology |
| D2     | Computer Science   |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Marks;
+-----+-----+-----+
| StudentID | Subject | Marks |
+-----+-----+-----+
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
+-----+-----+-----+
```

```
MySQL 8.0 Command Line Cll x + v
+-----+
| 101 | DBMS | 85 |
| 102 | DBMS | 78 |
| 103 | DBMS | 92 |
+-----+
15 rows in set (0.00 sec)

mysql> DROP DATABASE CollegeDB;
Query OK, 4 rows affected (0.22 sec)

mysql> CREATE DATABASE CollegeDB;
Query OK, 1 row affected (0.02 sec)

mysql> USE CollegeDB;
Database changed
mysql> DELETE FROM Marks;
ERROR 1146 (42S02): Table 'collegedb.marks' doesn't exist
mysql> SELECT * FROM Marks;
ERROR 1146 (42S02): Table 'collegedb.marks' doesn't exist
mysql> SHOW TABLES;
Empty set (0.07 sec)

mysql> CREATE TABLE Marks(
-> StudentID INT,
-> Subject VARCHAR(30),
-> Marks INT
-> );
Query OK, 0 rows affected (0.08 sec)

mysql> INSERT INTO Marks VALUES
-> (101,'DBMS',85),
-> (102,'DBMS',78),
-> (103,'DBMS',92);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> CREATE TABLE Student(
-> StudentID INT PRIMARY KEY,
-> Name VARCHAR(50),
-> DeptID VARCHAR(10)
-> );
Query OK, 0 rows affected (0.05 sec)
```

```
MySQL 8.0 Command Line Cll x + v

mysql> CREATE TABLE Department(
-> DeptID VARCHAR(10) PRIMARY KEY,
-> DeptName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.08 sec)

mysql> CREATE TABLE Marks(
-> StudentID INT,
-> Subject VARCHAR(30),
-> Marks INT
-> );
ERROR 1050 (42S01): Table 'marks' already exists
mysql> SHOW TABLES;
+-----+
| Tables_in_collegedb |
+-----+
| department           |
| marks                |
| student              |
+-----+
3 rows in set (0.02 sec)

mysql> SELECT * FROM student;
Empty set (0.01 sec)

mysql> SELECT * FROM department;
Empty set (0.00 sec)

mysql> DELETE FROM marks;
Query OK, 3 rows affected (0.05 sec)

mysql> DELETE FROM marks;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM marks;
Empty set (0.00 sec)

mysql> INSERT INTO marks VALUES
-> (101,'DBMS',85),
-> (102,'DBMS',78),
```

```

MySQL 8.0 Command Line CL
-> (103,'DBMS',92);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> DELETE FROM student;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO student VALUES
-> (101,'Aishwarya','D1'),
-> (102,'Rahul','D2'),
-> (103,'Priya','D1');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> DELETE FROM department;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO department VALUES
-> ('D1','Information Technology'),
-> ('D2','Computer Science');
Query OK, 2 rows affected (0.05 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> SELECT S.Name,
-> D.DeptName,
-> M.Marks
-> FROM student S
-> JOIN department D
-> ON S.DeptID = D.DeptID
-> JOIN marks M
-> ON S.StudentID = M.StudentID
-> WHERE M.Marks > 80;
+-----+-----+-----+
| Name | DeptName | Marks |
+-----+-----+-----+
| Aishwarya | Information Technology | 85 |
| Priya | Information Technology | 92 |
+-----+-----+-----+
2 rows in set (0.04 sec)

mysql> CREATE INDEX idx_student

```

```

mysql> CREATE INDEX idx_student
-> ON student(StudentID);
Query OK, 0 rows affected (0.14 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_marks
-> ON marks(StudentID);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE VIEW Student_Result AS
-> SELECT S.Name,
-> D.DeptName,
-> M.Marks
-> FROM student S
-> JOIN department D
-> ON S.DeptID = D.DeptID
-> JOIN marks M
-> ON S.StudentID = M.StudentID;
Query OK, 0 rows affected (0.04 sec)

mysql> SELECT * FROM Student_Result;
+-----+-----+-----+
| Name | DeptName | Marks |
+-----+-----+-----+
| Aishwarya | Information Technology | 85 |
| Rahul | Computer Science | 78 |
| Priya | Information Technology | 92 |
+-----+-----+-----+
3 rows in set (0.04 sec)

```

Performance Evaluation

Optimization Technique	Before Optimization	After Optimization
Subquery	Slower	Faster using JOIN
JOIN	Multiple nested operations	Direct table linking
INDEX	Full table scan	Faster searching using index
VIEW	Repeated SQL writing	Simplified query execution

Advantages of Optimization

- Reduces query execution time.
- Improves database performance.
- Minimizes repeated SQL code.
- Speeds up data retrieval.
- Enhances readability and maintainability.

Analysis

- Subqueries are suitable for simple filtering but may become slower with large datasets.
- JOINS are generally more efficient for combining related tables.
- Indexes improve search & join performance by allowing the database to locate rows.

9) Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem.

ANS

```
MySQL 8.0 Command Line Cli  x  +  v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 24
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE CollegeDB;
ERROR 1007 (HY000): Can't create database 'collegedb'; database exists
mysql> USE CollegeDB;
Database changed
mysql> CREATE TABLE student(
  -> StudentID INT PRIMARY KEY,
  -> Name VARCHAR(50),
  -> DeptID VARCHAR(10)
  -> );
ERROR 1050 (42S01): Table 'student' already exists
mysql> CREATE TABLE department(
  -> DeptID VARCHAR(10) PRIMARY KEY,
  -> DeptName VARCHAR(50)
  -> );
ERROR 1050 (42S01): Table 'department' already exists
mysql> CREATE TABLE marks(
  -> StudentID INT,
  -> Subject VARCHAR(30),
  -> Marks INT
  -> );
ERROR 1050 (42S01): Table 'marks' already exists
mysql> INSERT INTO student VALUES
  -> (101,'Aishwarya','D1'),
  -> (102,'Rahul','D2'),
  -> (103,'Priya','D1');
ERROR 1062 (23000): Duplicate entry '101' for key 'student.PRIMARY'
mysql> INSERT INTO department VALUES
  -> ('D1','Information Technology'),
  -> ('D2','Computer Science');
ERROR 1062 (23000): Duplicate entry 'D1' for key 'department.PRIMARY'
mysql> INSERT INTO marks VALUES
  -> (101,'DBMS',85),
  -> (102,'DBMS',78),
  -> (103,'DBMS',92);
Query OK, 3 rows affected (0.05 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM student;
+-----+-----+-----+
| StudentID | Name      | DeptID |
+-----+-----+-----+
| 101       | Aishwarya | D1     |
| 102       | Rahul     | D2     |
| 103       | Priya     | D1     |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM department;
+-----+-----+
| DeptID | DeptName      |
+-----+-----+
| D1     | Information Technology |
| D2     | Computer Science  |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM marks;
+-----+-----+-----+
| StudentID | Subject | Marks |
+-----+-----+-----+
| 101       | DBMS   | 85    |
| 102       | DBMS   | 78    |
| 103       | DBMS   | 92    |
| 101       | DBMS   | 85    |
| 102       | DBMS   | 78    |
| 103       | DBMS   | 92    |
+-----+-----+-----+
6 rows in set (0.00 sec)
```

```

MySQL 8.0 Command Line Cli  x  +  v
mysql> SELECT D.DeptName,
->         AVG(M.Marks) AS AverageMarks
-> FROM student S
-> JOIN department D
-> ON S.DeptID = D.DeptID
-> JOIN marks M
-> ON S.StudentID = M.StudentID
-> GROUP BY D.DeptName
-> HAVING AVG(M.Marks) >
-> (
-> SELECT AVG(Marks)
-> FROM marks
-> );

```

DeptName	AverageMarks
Information Technology	88.5000

1. JOIN Operation

FROM student S

JOIN department D

ON S.DeptID = D.DeptID

JOIN marks M

ON S.StudentID = M.StudentID

- Combines the student, department, and marks tables.

2. GROUP BY

GROUP BY D.DeptName;

Groups students according to their department.

Information Technology

- Aishwarya → 85
- Priya → 92

Average:

$$\frac{85 + 92}{2} = 88.5$$

Computer Science

- Rahul → 78

Average: 78

3. Aggregate Function

AVG(M.Marks)

Calculates the average marks for each department.

4. Nested Subquery

SELECT AVG(Marks)

FROM marks;

Overall average:

$$\frac{85 + 78 + 92}{3} = 85$$

5. HAVING Clause

HAVING AVG(M.Marks) > 85;

Only departments whose average is greater than 85 are displayed.

Performance Analysis

- JOIN efficiently combines related tables.
- GROUP BY groups records department-wise.
- AVG() computes departmental averages.
- Nested Subquery calculates the overall average.
- HAVING filters grouped results based on the aggregate value.

Expected Output

DeptName	AverageMarks
Information Technology	88.50

10) Given multiple relations, analyze the query requirements and decide whether to use Views, Joins, or Subqueries, justifying your choice with reasoning.

ANS

```
MySQL 8.0 Command Line Cli  ×  +  ▾

Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 25
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE CollegeDB;
ERROR 1007 (HY000): Can't create database 'collegedb'; database exists
mysql> USE CollegeDB;
Database changed
mysql> CREATE TABLE student(
  -> StudentID INT PRIMARY KEY,
  -> Name VARCHAR(50),
  -> DeptID VARCHAR(10)
  -> );
ERROR 1050 (42S01): Table 'student' already exists
mysql> CREATE TABLE department(
  -> DeptID VARCHAR(10) PRIMARY KEY,
  -> DeptName VARCHAR(50)
  -> );
ERROR 1050 (42S01): Table 'department' already exists
mysql> CREATE TABLE marks(
  -> StudentID INT,
  -> Subject VARCHAR(30),
  -> Marks INT
  -> );
ERROR 1050 (42S01): Table 'marks' already exists
mysql> INSERT INTO student VALUES
  -> (101, 'Aishwarya', 'D1'),
  -> (102, 'Rahul', 'D2'),
  -> (103, 'Priya', 'D1');
ERROR 1062 (23000): Duplicate entry '101' for key 'student.PRIMARY'
mysql> INSERT INTO department VALUES
  -> ('D1', 'Information Technology'),
```

```
MySQL 8.0 Command Line Cli x + v
-> ('D2','Computer Science');
ERROR 1062 (23000): Duplicate entry 'D1' for key 'department.PRIMARY'
mysql> INSERT INTO marks VALUES
-> (101,'DBMS',85),
-> (102,'DBMS',78),
-> (103,'DBMS',92);
Query OK, 3 rows affected (0.03 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM student;
+-----+-----+-----+
| StudentID | Name      | DeptID |
+-----+-----+-----+
| 101       | Aishwarya | D1     |
| 102       | Rahul    | D2     |
| 103       | Priya    | D1     |
+-----+-----+-----+
3 rows in set (0.05 sec)

mysql> SELECT * FROM department;
+-----+-----+
| DeptID | DeptName      |
+-----+-----+
| D1     | Information Technology |
| D2     | Computer Science  |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM marks;
+-----+-----+-----+
| StudentID | Subject | Marks |
+-----+-----+-----+
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
| 101       | DBMS    | 85    |
| 102       | DBMS    | 78    |
| 103       | DBMS    | 92    |
+-----+-----+-----+

35°C
Partly sunny
```

```
MySQL 8.0 Command Line Cli x + v
9 rows in set (0.04 sec)

mysql> SELECT S.Name,
-> D.DeptName,
-> M.Marks
-> FROM student S
-> JOIN department D
-> ON S.DeptID = D.DeptID
-> JOIN marks M
-> ON S.StudentID = M.StudentID
-> WHERE M.Marks > 80;
+-----+-----+-----+
| Name      | DeptName      | Marks |
+-----+-----+-----+
| Aishwarya | Information Technology | 85    |
| Aishwarya | Information Technology | 85    |
| Aishwarya | Information Technology | 85    |
| Priya     | Information Technology | 92    |
| Priya     | Information Technology | 92    |
| Priya     | Information Technology | 92    |
+-----+-----+-----+
6 rows in set (0.03 sec)

mysql> SELECT * FROM student;
+-----+-----+-----+
| StudentID | Name      | DeptID |
+-----+-----+-----+
| 101       | Aishwarya | D1     |
| 102       | Rahul    | D2     |
| 103       | Priya    | D1     |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM department;
+-----+-----+
| DeptID | DeptName      |
+-----+-----+
| D1     | Information Technology |
| D2     | Computer Science  |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM marks;
+-----+-----+-----+
| StudentID | Subject | Marks |
+-----+-----+-----+
| 101 | DBMS | 85 |
| 102 | DBMS | 78 |
| 103 | DBMS | 92 |
| 101 | DBMS | 85 |
| 102 | DBMS | 78 |
| 103 | DBMS | 92 |
| 101 | DBMS | 85 |
| 102 | DBMS | 78 |
| 103 | DBMS | 92 |
+-----+-----+-----+
9 rows in set (0.00 sec)

mysql> DELETE FROM marks;
Query OK, 9 rows affected (0.01 sec)

mysql> SELECT * FROM marks;
Empty set (0.00 sec)

mysql> INSERT INTO marks VALUES
-> (101,'DBMS',85),
-> (102,'DBMS',78),
-> (103,'DBMS',92);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM marks;
+-----+-----+-----+
| StudentID | Subject | Marks |
+-----+-----+-----+
| 101 | DBMS | 85 |
| 102 | DBMS | 78 |
| 103 | DBMS | 92 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT S.Name,
-> D.DeptName,
-> M.Marks
-> FROM student S
-> JOIN department D
-> ON S.DeptID = D.DeptID
-> JOIN marks M
-> ON S.StudentID = M.StudentID
-> WHERE M.Marks > 80;
+-----+-----+-----+
| Name | DeptName | Marks |
+-----+-----+-----+
| Aishwarya | Information Technology | 85 |
| Priya | Information Technology | 92 |
+-----+-----+-----+
2 rows in set (0.01 sec)

mysql>
```

35°C
Partly sunny

Search

Analysis and Comparison

Feature	JOIN	Subquery	VIEW
Purpose	Combine multiple tables	Filter records using another query	Save a frequently used query
Performance	Fast	Moderate	Fast for repeated access
Complexity	Suitable for multi-table queries	Suitable for simple filtering	Simplifies repeated queries
Data Retrieval	Multiple tables	Usually one table after filtering	Virtual table
Best Use	Reporting and analysis	Conditions and filtering	Security and abstraction

Justification

Why JOIN?

- Required because the query needs data from Student, Department, and Marks tables.
- Efficient for combining related data.

Why Subquery?

- Useful when filtering records based on results from another query.
- Easier for simple filtering tasks.

Why VIEW?

- Best for repeated queries.
- Improves security by exposing only selected columns.
- Simplifies query writing.

Final Decision

For this scenario, JOIN is the most appropriate choice because:

- The query requires data from multiple related tables.
- It provides better performance than nested subqueries for this type of retrieval.
- It is widely used in real-world database applications.

A VIEW is recommended if the same query is executed frequently, while a Subquery is suitable when only filtering based on another query is required.